

## Databricks Milestone 2 MCQs with Answers

**P001. A trainee installs Java to compile and run a program. Which component contains the Java compiler (javac)?**

- A. JDK
- B. JRE
- C. JVM only
- D. Garbage Collector

**Answer: A. JDK**

**P002. After a Java source file is compiled successfully, what does the compiler normally produce for the JVM to execute?**

- A. A native .exe file
- B. Java bytecode in a .class file
- C. A SQL script
- D. A compressed JAR only

**Answer: B. Java bytecode in a .class file**

**P003. A training app must store unique course codes and should automatically prevent duplicates. Which collection type best matches this requirement?**

- A. List
- B. Set
- C. Queue
- D. Array

**Answer: B. Set**

**P004. A Java method creates a local int variable and also creates a new Customer object. In the usual memory model, where is the object primarily allocated?**

- A. Stack only
- B. Heap
- C. CPU register only
- D. Source-code area

**Answer: B. Heap**

**P005. A method should accept a List of String values and prevent callers from accidentally adding Integer values at compile time. Which declaration is most suitable?**

- A. List values
- B. List values
- C. List values
- D. Object values

**Answer: C. List values**

**P006. A List receives the primitive value 25 using list.add(25). Why is this allowed?**

- A. Java converts the List into an int array
- B. Autoboxing converts int to Integer
- C. The compiler ignores the type
- D. Integer is a primitive type

**Answer: B. Autoboxing converts int to Integer**

**P007. A HashMap stores employee details using employeeid as the key. What happens when put() is called again with the same key and a new value?**

- A. A duplicate key is stored
- B. The existing value for that key is replaced
- C. The map becomes a Set
- D. The key is automatically deleted

**Answer: B. The existing value for that key is replaced**

**P008. A trainee must remove selected elements while iterating through an ArrayList and wants to avoid unsafe structural modification. Which approach is appropriate?**

- A. Call list.remove() from every foreach iteration
- B. Use Iterator and its remove() method after next()
- C. Convert the list to a String first
- D. Change the list reference to null

**Answer: B. Use Iterator and its remove() method after next()**

**P009. A condition is written as user != null && user.isActive(). If user is null, why does the expression avoid a NullPointerException from isActive()?**

- A. && evaluates the right side first
- B. && skips the right side when the left side is false
- C. isActive() is converted to static
- D. null is treated as an active user

**Answer: B. && skips the right side when the left side is false**

**P010. A BankAccount class keeps balance private and exposes deposit() and withdraw() methods. Which OOP principle is being applied?**

- A. Encapsulation
- B. Inheritance
- C. Method overloading
- D. Serialization

**Answer: A. Encapsulation**

**P011. Car extends Vehicle. What relationship does this represent?**

- A. Car has-a Vehicle
- B. Car is-a Vehicle
- C. Vehicle is a method of Car
- D. Car and Vehicle are unrelated

**Answer: B. Car is-a Vehicle**

**P012. A class contains print(int value) and print(String value). What concept does this demonstrate?**

- A. Overriding
- B. Overloading
- C. Abstraction only
- D. Constructor chaining

**Answer: B. Overloading**

**P013. A parent class defines calculateFee(), and a child class provides its own implementation with the same signature. Which concept is used?**

- A. Method overriding
- B. Method overloading
- C. Autoboxing
- D. Serialization

**Answer: A. Method overriding**

**P014. A child constructor must initialize fields defined by its parent class using the parent constructor. Which keyword is normally used?**

- A. this
- B. super
- C. final
- D. static

**Answer: B. super**

**P015. Several unrelated classes must guarantee that they provide a process() method, while each class can implement it differently. Which Java construct best expresses this contract?**

- A. Interface
- B. Private constructor
- C. Array
- D. Package only

**Answer: A. Interface**

**P016. An OrderService uses a PaymentGateway object to process payments, but OrderService is not a type of PaymentGateway. Which design relationship is most appropriate?**

- A. Inheritance
- B. Composition
- C. Overloading
- D. Casting

**Answer: B. Composition**

**P017. Inside a subclass, which expression refers to the current object, and which refers to the parent-class implementation?**

- A. this and super
- B. super and this
- C. static and final
- D. new and class

**Answer: A. this and super**

**P018. Vehicle v = new Car(); is followed by v.move(), and Car overrides move(). Which implementation executes?**

- A. Vehicle.move() always
- B. Car.move() because overridden instance methods use runtime dispatch
- C. Both methods execute automatically
- D. The code cannot compile

**Answer: B. Car.move() because overridden instance methods use runtime dispatch**

**P019. A program must read a plain text file containing customer names. Which type of stream is generally more suitable for text?**

- A. Character stream
- B. Byte stream only
- C. Object lock
- D. Thread pool

**Answer: A. Character stream**

**P020. What is the main benefit of try-with-resources when working with files or JDBC resources?**

- A. It automatically closes eligible resources
- B. It makes every exception checked
- C. It stores files in memory
- D. It prevents all runtime errors

**Answer: A. It automatically closes eligible resources**

**P021. Which Java class is appropriate for representing a calendar date without a time-of-day or timezone?**

- A. LocalDate
- B. LocalTimeZone
- C. FileDate
- D. ThreadDate

**Answer: A. LocalDate**

**P022. A program generates a simple text report containing formatted lines. Which class is convenient for writing textual output with print/println methods?**

- A. PrintWriter
- B. FileInputStream
- C. ObjectInputStream
- D. HashMap

**Answer: A. PrintWriter**

**P023. A trainee wants to check whether a product code follows a known text pattern. Which Java API is intended for regular-expression matching?**

- A. Pattern and Matcher
- B. LocalDate and Period
- C. List and Set
- D. Thread and Runnable only

**Answer: A. Pattern and Matcher**

**P024. A List should produce a new list containing only values greater than 100. Which Streams API operation performs the selection?**

- A. filter
- B. map only
- C. reduce only
- D. sorted only

**Answer: A. filter**

**P025. A program needs to visit files inside a directory tree using modern Java NIO APIs. Which operation is designed to stream paths recursively?**

- A. Files.walk(...)
- B. System.gc()
- C. Math.round(...)
- D. Thread.sleep(...)

**Answer: A. Files.walk(...)**

**P026. A Serializable User object contains a temporary sessionToken that should not be written during default Java serialization. Which keyword can mark that field?**

- A. transient
- B. volatile
- C. abstract
- D. extends

**Answer: A. transient**

**P027. A log line is "ERROR: payment failed". The regex "payment" should detect that word anywhere inside the line. Which Matcher method is the better fit?**

- A. matches(), because it requires the whole line to match
- B. find(), because it searches for a matching subsequence
- C. replaceAll(), because it returns Boolean
- D. group(), before any match attempt

**Answer: B. find(), because it searches for a matching subsequence**

**P028. Which Scala keyword creates a reference that cannot be reassigned after initialization?**

- A. val
- B. var
- C. def
- D. new

**Answer: A. val**

**P029. What is the typical behavior of Scala immutable List operations such as map?**

- A. They modify the original list in place
- B. They return a new collection value
- C. They always return Unit
- D. They require a database

**Answer: B. They return a new collection value**

**P030. A Scala program needs to return a customer ID and customer name together without defining a new class. Which simple structure can hold both values?**

- A. Tuple
- B. Thread
- C. Exception
- D. RDD action

**Answer: A. Tuple**

**P031. Scala sees `val total = 125.50`. What does type inference mean in this statement?**

- A. The programmer must write the type later
- B. Scala determines an appropriate type from the assigned value
- C. The value has no type
- D. The variable becomes a String automatically

**Answer: B. Scala determines an appropriate type from the assigned value**

**P032. A status code can be NEW, PAID, or CANCELLED and each value needs a different result. Which Scala feature provides clear case-based branching?**

- A. match with case clauses
- B. Only while loops
- C. JDBC commit
- D. RDD cache

**Answer: A. match with case clauses**

**P033. A `List(2,4,6)` should become `List(4,8,12)` without changing the original list. Which operation is most suitable?**

- A. map
- B. filter
- C. foreach only
- D. dropWhile only

**Answer: A. map**

**P034. A Scala program needs an indexed collection whose elements can be updated by position. Which basic collection is appropriate?**

- A. Array
- B. Immutable Tuple only
- C. Option
- D. Try

**Answer: A. Array**

**P035. Which expression is a valid simple Scala lambda that doubles an integer x?**

- A. `x => x * 2`
- B. `x = x * 2` only
- C. `double(x)` class
- D. case object `x * 2`

**Answer: A. `x => x * 2`**

**P036. A match case should classify only positive amounts above 1000 as "High". Which form best expresses this additional condition?**

- A. `case x if x > 1000 => "High"`
- B. `case x = 1000 => "High"`

- C. if case x then 1000
- D. case => x > 1000 only

**Answer: A. case x if x > 1000 => "High"**

**P037. Which of the following is an RDD transformation that returns a new RDD?**

- A. map
- B. count
- C. collect
- D. first

**Answer: A. map**

**P038. Which RDD operation is an action that returns the number of elements?**

- A. count
- B. filter
- C. map
- D. flatMap

**Answer: A. count**

**P039. What does lazy evaluation mean for Spark RDD transformations?**

- A. They run immediately when written
- B. Spark records transformations and executes them when an action requires results
- C. They never execute
- D. Only the driver can use them

**Answer: B. Spark records transformations and executes them when an action requires results**

**P040. An RDD partition is lost after an executor failure. How can Spark often rebuild it?**

- A. Using the recorded lineage of transformations
- B. Only from a manual backup file
- C. By restarting the operating system
- D. By converting it to a Java List

**Answer: A. Using the recorded lineage of transformations**

**P041. A pair RDD contains (product, amount) records. Which operation is commonly used to combine amounts for the same product efficiently?**

- A. reduceByKey
- B. collect before aggregation
- C. mapPartitions only
- D. repartition to one partition first

**Answer: A. reduceByKey**

**P042. A small Scala collection already exists on the driver for a training example. Which Spark method can distribute it as an RDD?**

- A. sc.parallelize(collection)
- B. sc.collect(collection)
- C. df.show(collection)
- D. spark.stop(collection)

**Answer: A. sc.parallelize(collection)**

**P043. The same expensive RDD is reused in several actions during one application. Which Spark feature can avoid recomputing it each time?**

- A. persist/cache
- B. collect then delete the RDD
- C. Rename the RDD
- D. Convert every element to String

**Answer: A. persist/cache**

**P044. Why is map usually described as a narrow transformation?**

- A. Each output partition can be computed from a limited input partition without a full data shuffle
- B. It always reduces partitions to one
- C. It broadcasts every record
- D. It runs only on the driver

**Answer: A. Each output partition can be computed from a limited input partition without a full data shuffle**

**P045. A 500 GB RDD is processed successfully, but the driver fails when collect() is called. What is the main reason?**

- A. collect() attempts to bring all elements to the driver
- B. collect() deletes executor memory
- C. collect() disables Spark
- D. collect() changes every value to null

**Answer: A. collect() attempts to bring all elements to the driver**

**P046. Which Scala utility is commonly used in simple examples to read text from a file?**

- A. scala.io.Source
- B. scala.collection.Set
- C. SparkSession.stop
- D. ThreadLocal

**Answer: A. scala.io.Source**

**P047. What does Option help represent in Scala?**

- A. A value that may be present or absent
- B. Only a mutable list
- C. A network socket
- D. A thread lock

**Answer: A. A value that may be present or absent**

**P048. A file-reading operation may throw an exception. Which Scala type can capture success or failure as a value?**

- A. Try
- B. Tuple2 only
- C. Array
- D. RDD partition

**Answer: A. Try**

**P049. A Source is opened to read a text file. What should the program ensure after reading is complete?**

- A. The Source is closed
- B. The JVM is always restarted
- C. The file is converted into an RDD
- D. The path is stored in a Set

**Answer: A. The Source is closed**

**P050. A validation function should return either an error message or a valid Order. Which Scala type naturally models two alternative result types?**

- A. Either
- B. Array only
- C. Unit
- D. Thread

**Answer: A. Either**

**P051. A trainee opens Source.fromFile(path). Which method is commonly used to iterate through file lines?**

- A. getLines()
- B. collect()
- C. reduceByKey()
- D. commit()

**Answer: A. getLines()**

**P052. A Try[Int] should return 0 when parsing fails. Which concept is designed to provide a fallback for a failed Try?**

- A. recover or getOrElse
- B. RDD checkpoint only
- C. Thread interrupt
- D. Pattern.compile only

**Answer: A. recover or getOrElse**

**P053. A Map lookup returns Option[String]. Which approach safely provides "Unknown" when no value exists?**

- A. option.getOrElse("Unknown")
- B. option.get always
- C. option.close()
- D. option.collectAsMap()

**Answer: A. option.getOrElse("Unknown")**

**P054. A program must open a file path and then parse an integer from its contents, and either step may fail. Why can wrapping the operations in Try be useful?**

- A. It lets success/failure flow through composable operations instead of requiring unchecked assumptions
- B. It guarantees the file exists
- C. It converts every exception to null
- D. It makes all I/O asynchronous

**Answer: A. It lets success/failure flow through composable operations instead of requiring unchecked assumptions**

**P055. Which DataFrame method is commonly used to inspect how Spark plans to execute a query?**

- A. explain()
- B. showSchemaOnly()
- C. collectPlan()
- D. savePlan()

**Answer: A. explain()**

**P056. When a transformation can be expressed using built-in Spark SQL functions, which approach is generally preferred?**

- A. Use built-in functions
- B. Always write a custom UDF
- C. Collect data to the driver
- D. Convert everything to text files first

**Answer: A. Use built-in functions**

**P057. Why is Parquet often efficient for analytical queries that select only a few columns?**

- A. It is columnar and can avoid reading unnecessary columns
- B. It stores every row as an image
- C. It requires collect() first
- D. It runs only in memory

**Answer: A. It is columnar and can avoid reading unnecessary columns**

**P058. A Spark job needs to read a table from a relational database. Which Spark data source is appropriate?**

- A. JDBC
- B. Socket text only
- C. RDD checkpoint
- D. LocalDate

**Answer: A. JDBC**

**P059. A trainee wants to know which join operator Spark actually selected after planning. Which plan is most relevant?**

- A. Physical plan
- B. Only the source file name
- C. DataFrame schema only
- D. Java heap dump

**Answer: A. Physical plan**

**P060. A Parquet dataset is partitioned by event\_date. A query filters event\_date to one day. What optimization can avoid scanning unrelated date folders?**

- A. Partition pruning
- B. Autoboxing
- C. Thread synchronization
- D. Object serialization

**Answer: A. Partition pruning**

**P061. A Parquet table has 150 columns, but a report selects only 5. Which optimization reduces unnecessary column I/O?**

- A. Column pruning
- B. Deadlock detection
- C. JDBC rollback
- D. Scala pattern guards

**Answer: A. Column pruning**

**P062. A very large JDBC table is currently read through a single input partition. At a conceptual level, what can improve source-side parallel reading when a suitable numeric key exists?**

- A. Configure partitioned JDBC reading using a partition column and multiple partitions
- B. Call collect() immediately
- C. Increase only the number of DataFrame columns
- D. Write the table to a Java array first

**Answer: A. Configure partitioned JDBC reading using a partition column and multiple partitions**

**P063. Two implementations return the same cleaned customer name. One uses built-in trim/upper functions; the other uses a custom UDF. Why is the built-in expression usually easier for Spark to optimize?**

- A. Its transformation semantics are visible to Spark SQL planning
- B. It always uses one partition
- C. It automatically caches all results
- D. It executes only on the driver

**Answer: A. Its transformation semantics are visible to Spark SQL planning**

**P064. Which collection operation transforms every element and returns a new collection?**

- A. map
- B. filter
- C. exists
- D. foreach only

**Answer: A. map**

**P065. Which Scala collection operation keeps only elements that satisfy a Boolean condition?**

- A. filter
- B. map
- C. flatten only
- D. zip only

**Answer: A. filter**

**P066. When each input element can produce zero or more output elements and the final result should be flat, which operation is especially useful?**

- A. flatMap
- B. head
- C. size
- D. reverse only

**Answer: A. flatMap**

**P067. A list of transactions must first remove invalid records and then normalize the remaining records. Which chain best matches the required order?**

- A. transactions.filter(valid).map(normalize)
- B. transactions.map(normalize).foreach(valid)
- C. transactions.foreach(valid).filter(normalize)
- D. transactions.reduce(valid).map(normalize)

**Answer: A. transactions.filter(valid).map(normalize)**

**P068. A reusable select function should accept the rule to apply from the caller. Which parameter type represents a predicate for elements of type A?**

- A. A => Boolean
- B. String only
- C. Boolean => A always
- D. Unit => Unit only

**Answer: A. A => Boolean**

**P069. A tax function should return the same result for the same transaction and tax rate every time. Which design best supports this?**

- A. Pass required values as parameters and return the result without changing shared state
- B. Read a changing global tax rate
- C. Update a shared mutable variable during calculation
- D. Write to a global buffer before returning

**Answer: A. Pass required values as parameters and return the result without changing shared state**

**P070. A method receives an immutable List and must remove invalid values. What should it do?**

- A. Return a new filtered list
- B. Cast and mutate the original list
- C. Store it globally and modify it later
- D. Return Unit and change external state

**Answer: A. Return a new filtered list**

**P071. Only active customers should contribute their verified contacts, and each customer can have many contacts. Which chain best fits?**

- A. customers.filter(\_.active).flatMap(\_.contacts).filter(\_.verified)
- B. customers.map(\_.contacts) only
- C. customers.foreach(\_.contacts).map(\_.verified)
- D. customers.head.contacts only

**Answer: A. customers.filter(\_.active).flatMap(\_.contacts).filter(\_.verified)**

**P072. A pipeline must validate records, convert valid ones into objects, and emit zero or more results per object without mutable buffers. Which sequence best expresses the flow?**

- A. filter followed by map followed by flatMap
- B. foreach followed by global ArrayBuffer updates
- C. collect everything first, then mutate a shared list
- D. Use only while loops with shared state

**Answer: A. filter followed by map followed by flatMap**

**P073. A CSV contract already defines exact column names and types. What should Spark use when creating the DataFrame for consistent interpretation?**

- A. An explicit schema
- B. A random schema each run
- C. Only a local List
- D. No column names

**Answer: A. An explicit schema**

**P074. For structured rows with named columns, filters, joins, and aggregations, which Spark abstraction is usually more suitable?**

- A. DataFrame
- B. Plain local Array only
- C. Driver-side HashMap only
- D. Thread

**Answer: A. DataFrame**

**P075. What does inferSchema do when reading supported data such as CSV?**

- A. Spark examines data to guess column data types
- B. It encrypts the file
- C. It caches all rows permanently
- D. It creates a JDBC connection

**Answer: A. Spark examines data to guess column data types**

**P076. Two DataFrames contain the same fields in different column orders. Which operation can append rows by matching column names rather than positions?**

- A. unionByName
- B. join only
- C. collect
- D. groupByKey on an RDD

**Answer: A. unionByName**

**P077. df2 is missing one column that exists in df1. Which option allows unionByName to fill the missing column with null?**

- A. allowMissingColumns = true
- B. inferSchema = false
- C. header = true
- D. cache = true

**Answer: A. allowMissingColumns = true**

**P078. Two versions produce the same output. One uses DataFrame column expressions; the other converts rows to arbitrary objects and opaque functions. Which version exposes more relational information to Spark?**

- A. The DataFrame expression version
- B. The local object version
- C. Both always expose identical semantics
- D. The version that calls collect() first

**Answer: A. The DataFrame expression version**

**P079. A large DataFrame result must remain distributed and be written to shared storage. Which approach is appropriate?**

- A. Use DataFrameWriter to write the distributed result
- B. collect() everything and use one local file writer
- C. Convert it to one Scala List first
- D. Print every row on the driver

**Answer: A. Use DataFrameWriter to write the distributed result**

**P080. A schema declares amount as Decimal, but a source record contains invalid text. Which statement is most accurate?**

- A. The schema states the intended type, but malformed input still depends on parsing and validation behavior
- B. The schema guarantees all business values are valid
- C. Spark always repairs every malformed value
- D. Invalid text automatically becomes a valid decimal

**Answer: A. The schema states the intended type, but malformed input still depends on parsing and validation behavior**

**P081. A workload mainly performs projections, filters, joins, and aggregations on structured order records. The team wants Spark to inspect these operations for optimization. Why is a DataFrame a stronger fit than an RDD of opaque domain objects?**

- A. DataFrames express named-column relational operations that Spark SQL can analyze
- B. RDDs cannot be distributed
- C. DataFrames always avoid shuffles
- D. RDDs execute only on the driver

**Answer: A. DataFrames express named-column relational operations that Spark SQL can analyze**

**P082. Which Spark API provides DataFrame/Dataset-style processing for streaming data?**

- A. Structured Streaming
- B. Java Serialization
- C. RDD collect
- D. JDBC only

**Answer: A. Structured Streaming**

**P083. Which statement best describes DStreams in modern Spark training?**

- A. They are the older RDD-based streaming abstraction
- B. They are Parquet files
- C. They are Java interfaces
- D. They are database indexes

**Answer: A. They are the older RDD-based streaming abstraction**

**P084. What is the basic purpose of a checkpoint location in a streaming query?**

- A. Persist recovery/progress information on durable storage
- B. Increase the number of Java constructors
- C. Convert JSON to XML
- D. Replace every source with a socket

**Answer: A. Persist recovery/progress information on durable storage**

**P085. A trainer wants a simple live demo where trainees type text into a TCP socket and see Spark process it. Is a socket source reasonable for this purpose?**

- A. Yes, for demonstration/testing
- B. No, sockets cannot carry text
- C. Only if the data is first written to JDBC
- D. Only for batch jobs

**Answer: A. Yes, for demonstration/testing**

**P086. New CSV files are dropped into a watched folder after they are completely written. Which Structured Streaming source can process newly arriving files?**

- A. File source with readStream and a defined schema
- B. collect() on a static List
- C. Java FileOutputStream only
- D. LocalDate API

**Answer: A. File source with readStream and a defined schema**

**P087. A business wants sales totals for each 10-minute interval based on event timestamps. Which concept is appropriate?**

- A. Event-time window aggregation
- B. JVM heap only
- C. Method overloading
- D. Object serialization

**Answer: A. Event-time window aggregation**

**P088. Events may arrive several minutes late, but Spark should eventually stop retaining old event-time state. Which feature addresses accepted lateness and state cleanup?**

- A. Watermark
- B. Autoboxing
- C. JDBC commit
- D. RDD parallelize

**Answer: A. Watermark**

**P089. A streaming query restarts after a driver failure. Which design is important if it should continue using previously recorded progress?**

- A. Restart with the same durable checkpoint location when the query is compatible
- B. Use only executor cache
- C. Always create a brand-new checkpoint
- D. Store offsets only in a local mutable variable

**Answer: A. Restart with the same durable checkpoint location when the query is compatible**

**P090. An upstream process writes large files slowly. How can it reduce the chance that Spark sees partially written input files in a watched folder?**

- A. Write elsewhere and atomically move/rename completed files into the watched location
- B. Let Spark read while files are still being written
- C. Delete files before they finish
- D. Store filenames only in driver memory

**Answer: A. Write elsewhere and atomically move/rename completed files into the watched location**

**P091. A query uses event-time windows and allows events up to 10 minutes late. Why is a 5-minute processing trigger not a replacement for a 10-minute watermark?**

- A. The trigger controls how often processing runs, while the watermark relates to event-time lateness/state
- B. The trigger changes event timestamps
- C. The watermark only controls executor count
- D. They are identical settings

**Answer: A. The trigger controls how often processing runs, while the watermark relates to event-time lateness/state**

**P092. Two threads update the same shared counter and some increments are lost. Which concurrency problem is this?**

- A. Race condition
- B. Method overloading
- C. Schema inference
- D. RDD lineage

**Answer: A. Race condition**

**P093. Thread A holds Lock1 and waits for Lock2 while Thread B holds Lock2 and waits for Lock1 indefinitely. What problem is illustrated?**

- A. Deadlock
- B. Autoboxing
- C. Serialization
- D. Column pruning

**Answer: A. Deadlock**

**P094. Creating an order and reducing inventory must either both succeed or both fail. Which JDBC transaction approach is appropriate?**

- A. Disable auto-commit, commit after both succeed, rollback on failure
- B. Commit after the first statement regardless of later errors
- C. Use a different connection for each statement with no coordination
- D. Call collect() after each SQL statement

**Answer: A. Disable auto-commit, commit after both succeed, rollback on failure**

**P095. Two fields must be updated together so readers never observe a half-updated state. What concept can protect the compound operation?**

- A. Use the same shared lock/synchronization around related updates and reads
- B. Use two unrelated local locks created per call
- C. Mark only one field static
- D. Use HashSet instead of synchronization

**Answer: A. Use the same shared lock/synchronization around related updates and reads**

**P096. Many threads need to read and update a shared map. Which Java collection is designed for concurrent access better than a plain HashMap without external synchronization?**

- A. ConcurrentHashMap
- B. ArrayList
- C. TreeSet only
- D. StringBuilder

**Answer: A. ConcurrentHashMap**

**P097. A service wants to represent a result that will be completed asynchronously and then apply another non-blocking step. Which Java API is designed for this style?**

- A. CompletableFuture
- B. FileInputStream
- C. LocalDate
- D. HashSet

**Answer: A. CompletableFuture**

**P098. A listener list is read very frequently and modified only occasionally. Which concurrent list is known for snapshot-style iteration with relatively expensive writes?**

- A. CopyOnWriteArrayList
- B. LinkedHashMap
- C. ArrayDeque
- D. String[]

**Answer: A. CopyOnWriteArrayList**

**P099. Why do server applications commonly use a JDBC connection pool?**

- A. To reuse a limited set of database connections instead of opening a new physical connection for every request
- B. To convert SQL into Scala
- C. To guarantee every query is fast

D. To replace database transactions

**Answer: A. To reuse a limited set of database connections instead of opening a new physical connection for every request**

**P100. Transfers can lock two Account objects. One thread locks account 10 then 20, while another may lock 20 then 10. Which simple design reduces deadlock risk while keeping per-account locks?**

A. Acquire required account locks in a consistent deterministic order such as ascending account ID

B. Always lock the source first regardless of ID

C. Create a brand-new lock object for each transfer

D. Hold the first lock while repeatedly sleeping forever

**Answer: A. Acquire required account locks in a consistent deterministic order such as ascending account ID**